





UNIVERSITÉ DE MONTPELLIER

Faculté des Sciences — Master Informatique

Mémoire de fin d'études

Master 2 Informatique — Parcours Génie Logiciel (GL)

Réduction de la dette technique dans un environnement de production

Mémoire rédigé à l'issue de deux années d'alternance

Travaux réalisés dans le cadre d'un contrat d'apprentissage au sein de
Free Mobile et Beemo Technologie.

Présenté par

Jérémy Hurel

Encadrement

David Desmarest — *Architecte Logiciel, tuteur entreprise*
Mickaël Montassier — *tuteur universitaire*

Année universitaire 2024 – 2026

Remerciements

Je tiens en premier lieu à remercier David Desmarest pour m'avoir donné l'opportunité de réaliser mon alternance, d'abord chez Free Mobile, puis pour m'avoir proposé un poste chez Beemo Technologie afin de la mener à son terme. Je le remercie pour sa pédagogie tout au long de ces deux années et pour tout ce qu'il m'a appris dans le domaine du génie logiciel, en particulier au travers des missions qu'il m'a confiées et qui m'ont permis de progresser.

Je souhaite également remercier Michael Montassier, qui a su se rendre joignable en toutes circonstances pour répondre à mes questions, notamment lorsque s'est posée celle de quitter ou non Free Mobile.

Enfin, je remercie l'Université de Montpellier d'avoir mis en place son *job dating*, qui m'a permis d'obtenir mon poste chez Free Mobile, ainsi que l'ensemble des collègues que j'ai côtoyés aussi bien chez Free que chez Beemo. Tous se sont montrés ouverts et disponibles pour répondre à mes questions, sans jamais faire de différence liée à mon statut d'alternant.

Résumé

[Résumé à rédiger.] Ce mémoire retrace une alternance menée au sein de deux entreprises — Free Mobile puis Beemo Technologie — autour d'un fil conducteur commun : la réduction de la dette technique et les enjeux business qui lui sont associés. *[...]*

Mots-clés : dette technique, tests automatisés, non-régression, intégration continue, rétro-ingénierie, qualité logicielle.

Abstract. *[Traduction anglaise du résumé — facultatif mais apprécié.]*

Table des matières

Remerciements **i**

Résumé **ii**

1 Introduction **1**
1.1 Contexte de l'alternance 1
1.2 Le fil conducteur : la dette technique 1
1.3 Problématique 1
1.4 Plan du mémoire 1

2 Présentation des entreprises **2**
2.1 Free Mobile 2
2.2 Beemo Technologie 3
2.3 Positionnement des missions 3

3 Environnement technique **4**
3.1 Langages et bibliothèques 4
3.2 Stockage et transport de données 5
3.3 Outils de build et de compilation 5
3.4 Gestion de version et intégration continue 5
3.5 Méthodologie et gestion de projet 5

4 Appropriation scientifique : la dette technique et ses enjeux business **7**
4.1 Définition et métaphore 7
4.2 Typologie de la dette technique 7
4.3 Enjeux business 7
4.4 Retour d'expérience : stratégies d'adressage et dimension sociale 8
4.4.1 Une même logique d'adressage incrémental 8
4.4.2 Des dettes de nature différente 8
4.4.3 L'enjeu social du traitement de la dette 9
4.5 Leviers de réduction 9

4.6 Grille de lecture des missions 9

5 Tâche 1 — Outil de duplication de configuration Grafana 10
5.1 Contexte et dette technique initiale 10
5.2 Objectif de la mission 10
5.3 Conception et choix techniques 11
5.4 Réalisation 11
5.5 Difficultés rencontrées 12
5.6 Résultats et bénéfice business 12

6 Tâche 2 — Maîtrise du coût de compilation du Valo Engine 13
6.1 Contexte et dette technique initiale 13
6.2 Objectif de la mission 13
6.3 Conception et choix techniques 14
6.4 Réalisation 14
6.5 Difficultés rencontrées 15
6.6 Résultats et bénéfice business 15

7 Tâche 3 — Framework de test d'intégration et de non-régression 16
7.1 Contexte et dette technique initiale 16
7.2 Objectif de la mission 16
7.3 Conception et choix techniques 17
7.4 Réalisation 18
7.5 Difficultés rencontrées 18
7.6 Résultats et bénéfice business 19

8 Tâche 4 — Outillage du framework : extension VS Code (LSP et débogueur) 20
8.1 Contexte et dette technique initiale 20
8.2 Objectif de la mission 20
8.3 Conception et choix techniques 21
8.4 Réalisation 21
8.5 Difficultés rencontrées 22
8.6 Résultats et bénéfice business 23

9 Tâche 5 — Portage du framework vers les forks legacy du Valo Engine 24
9.1 Contexte et dette technique initiale 24
9.2 Objectif de la mission 25
9.3 Conception : factorisation et paramétrage 25
9.4 Réalisation 25
9.5 Difficultés rencontrées 26
9.6 Résultats et bénéfice business 26

10 Tâche 6 — Rétro-ingénierie pour la compilation avec un SDK 28
10.1 Contexte et dette technique initiale . . . 28
10.2 Objectif de la mission . . . 29
10.3 Démarche de rétro-ingénierie . . . 29
10.4 Réalisation . . . 29
10.5 Difficultés rencontrées . . . 30
10.6 Résultats et bénéfice business . . . 30

11 Tâche 7 — Développement d'une fonctionnalité en code de production 31
11.1 Contexte et besoin . . . 31
11.2 Objectif de la mission . . . 32
11.3 Conception et intégration . . . 32
11.4 Réalisation . . . 33
11.5 Difficultés rencontrées . . . 34
11.6 Résultats et bénéfice business . . . 34

12 Conclusion 35
12.1 Bilan fonctionnel . . . 35
12.2 Retour sur le fil rouge : la dette technique . . . 35
12.3 Apports de l'alternance . . . 35
12.4 Perspectives . . . 35

Glossaire des acronymes 36

Glossaire 37

A Annexes 39
A.1 Diagramme de Gantt . . . 39
A.2 Extrait de code choisi . . . 39

Chapitre 1

Introduction

1.1 Contexte de l'alternance

[À rédiger.]

1.2 Le fil conducteur : la dette technique

[À rédiger : poser la problématique et les enjeux business.]

1.3 Problématique

[À rédiger.]

1.4 Plan du mémoire

[À rédiger.]

Chapitre 2

Présentation des entreprises

2.1 Free Mobile

Free Mobile est un opérateur de télécommunications du groupe Iliad, fondé par Xavier Niel. L'entreprise est notamment connue pour être entrée sur le marché de la téléphonie mobile de manière agressive, en particulier avec son forfait à deux euros. Au-delà de ses offres commerciales, Free Mobile fait partie des rares opérateurs à maîtriser l'ensemble de leurs logiciels en interne : tout leur cœur de réseau y est développé, de même que la valorisation, et l'entreprise dispose en outre d'une section R&D dédiée à ses propres équipements matériels.

J'ai intégré l'équipe Valo, rattachée au Core SI. Cette équipe a la charge du logiciel de valorisation, qui calcule la valeur de l'ensemble des appels passés en téléphonie mobile sur les différents réseaux.

À mon arrivée, ce logiciel se trouvait dans un contexte de dette technique élevée : l'ingénieur qui en assurait la maintenance avait quitté l'entreprise, et plusieurs règles de génie logiciel n'avaient pas été respectées — une partie des sources n'étant par exemple pas versionnée sous Git. L'équipe, montée et dirigée par David Desmarest, s'est vu confier une mission de reprise de contrôle de ce logiciel.

Mon rôle y était transverse au travail de l'équipe de développement : supporter celle-ci en produisant des outils internes, au premier rang desquels le framework de test détaillé au chapitre 7.

2.2 Beemo Technologie

Beemo Technologie est une entreprise d'une vingtaine d'années spécialisée dans la sauvegarde de données. Sa particularité est de ne pas se limiter à un logiciel installé sur une machine du client, mais de proposer une solution clé en main : des boîtiers (*box*) déployés au sein de l'entreprise — voire chez son dirigeant — qui assurent la sauvegarde et la duplication des données, et donc leur redondance en cas de perte.

L'entreprise compte une trentaine d'employés et dispose de clients et de partenaires partout dans le monde ; elle se développe actuellement sur le continent africain. Une partie des effectifs est dédiée à une *hotline*, reconnue pour sa réactivité et son efficacité dans la résolution des problèmes. Beemo se distingue enfin par le caractère souverain de sa solution de sauvegarde, un atout particulièrement recherché dans le contexte actuel.

J'ai intégré l'équipe de développement à la suite d'une proposition de David Desmarest, qui, après avoir quitté Free Mobile, en avait pris la direction. À la différence de Free Mobile, où mon rôle était transverse et orienté support, j'occupais ici un poste de développeur travaillant directement sur le fonctionnel. Le contexte restait toutefois marqué par la dette technique : l'enjeu était de demeurer productif face au marché — c'est-à-dire de continuer à livrer de nouvelles fonctionnalités — tout en résorbant progressivement cette dette.

2.3 Positionnement des missions

Entreprise	**Tâche**	**Objectif au regard de la dette technique**
Free Mobile	Duplication Grafana	Fiabiliser la réplication de la supervision
Free Mobile	Compilation C++	Maîtriser le coût de compilation (CPU/RAM)
Free Mobile	Framework de test	Industrialiser la non-régression
Free Mobile	Extension VsCode	Améliorer la productivité des développeurs
Free Mobile	Portage multi-forks	Mutualiser la qualité sur les variantes
Beemo	Rétro-ing. SDK	Débloquer une compilation existante
Beemo	Dév. feature	Livrer sans accroître la dette

Table 2.1 – Vue d'ensemble des missions.

Chapitre 3

Environnement technique

3.1 Langages et bibliothèques

Les environnements techniques de Free Mobile et de Beemo se sont révélés suffisamment proches pour que cette similarité ait précisément motivé ma candidature : c'est ce type d'environnement qui m'intéressait. Dans les deux entreprises, le logiciel principal est écrit en C++, et gravite autour de lui un ensemble d'outils externes développés dans différents langages. Pour tout ce qui relève du *scripting*, Python est très largement utilisé, le plus souvent appelé depuis des scripts *batch* ; on trouve par ailleurs des Application Programming Interface (API) écrites dans des langages moins orientés haute performance que le C++.

Le choix du C++ comme langage principal tient d'abord à l'attrait du langage lui-même : il a beaucoup évolué et propose aujourd'hui une bibliothèque standard et des fonctionnalités permettant d'écrire un code conforme aux recommandations issues des dernières avancées du génie logiciel. Étant compilé, il offre en outre des performances élevées. Chez Free Mobile, où un gros volume de données devait être traité, ce point était impératif ; chez Beemo, où les *box* ont vocation à sauvegarder plusieurs téraoctets de données, de bonnes performances restent nécessaires pour garantir une expérience utilisateur correcte.

Le choix de Python répond, lui, à une logique de facilité et de portabilité. L'absence d'étape de compilation est une force : elle dispense d'une chaîne de Continuous Integration (intégration continue) (CI)/Continuous Delivery / Deployment (CD) complexe pour produire les nombreux outils internes utiles au quotidien, et permet ainsi de gagner en productivité. La même logique vaut pour les autres langages employés sur les logiciels que l'on pourrait qualifier de plus proches du *front* : on n'y recherche pas tant des performances élevées qu'un livrable facilement déployable et pouvant être mis à jour sans difficulté majeure.

3.2 Stockage et transport de données

Chez Free Mobile, un effort de standardisation des outils de stockage et de transfert de données a été mené. On y retrouve ainsi une base de données MySQL pour les usages historiques liés aux utilisateurs, ainsi qu'Apache Kafka pour produire des flux de données et remonter des informations vers d'autres équipes. Le monitoring s'appuie sur Prometheus, qui permet de collecter des métriques directement sur les différents logiciels en exploitation.

Le stockage des Call Detail Record (enregistrement détaillé d'appel) (CDR) — les données générées par les équipements téléphoniques, terme standard du domaine des télécoms — repose sur une base NoSQL, Cassandra. Cette dernière présentant toutefois certaines limitations, ClickHouse a été introduit en complément : il s'agit d'une base de données relationnelle orientée colonne, qui offre d'excellentes performances sur les requêtes d'agrégation et se prête donc particulièrement bien au calcul de statistiques sur de gros volumes de données.

3.3 Outils de build et de compilation

[À rédiger.]

3.4 Gestion de version et intégration continue

[À rédiger.]

3.5 Méthodologie et gestion de projet

Chez Free Mobile, l'organisation du travail suit une méthode *agile*, structurée en *sprints* et rythmée par des points quotidiens (*daily meetings*). La durée des sprints était ajustée à la nature des logiciels maintenus par l'équipe : la partie cœur, écrite en C++, ainsi que les outils liés au traitement des CDR étaient développés sur des sprints de trois semaines, tandis que la partie API et interface web, dont les besoins différaient, suivait des cycles plus courts de deux semaines.

Le rituel central était le point quotidien d'une quinzaine de minutes, tenu en début de journée — autour de huit heures quarante-cinq, selon les heures d'arrivée de chacun — avant le début effectif de la journée de travail. Ce rendez-vous court permettait à chaque membre de l'équipe de partager son avancement et d'identifier rapidement les éventuels points de blocage.

Mon positionnement dans cette organisation était particulier : mes développements n'étaient

pas intégrés aux sprints. Je travaillais en effet sur des outils annexes dont la production ne dépendait pas directement, même s'ils aidaient les développeurs à produire un code plus sûr et à gagner en confiance lors des *releases*. Ce statut « hors sprint » m'a offert un réel confort de développement, en me dégageant de la pression liée aux échéances de livraison.

Chez Beemo Technologie, l'organisation repose également sur une méthode *agile*, mais déclinée de façon plus complète : on y retrouve l'ensemble des étapes du processus. Le point quotidien s'y tient à onze heures quarante-cinq, et s'y ajoute une phase de raffinement (*refinement*) au cours de laquelle les tickets sont précisés avant le démarrage de leur développement — une pratique qui n'était pas spécifiquement mise en œuvre chez Free Mobile, où l'on se montrait plus flexible sur ce point. Chaque fin de sprint, d'une durée de deux à trois semaines, donne par ailleurs lieu à une rétrospective.

Au-delà de ces nuances, il n'existe pas de différence majeure entre les deux équipes : la méthode agile correspond bien aux besoins du développement dans les deux contextes. En revanche, mon rôle y était sensiblement différent de celui que j'occupais chez Free Mobile. Mes développements ayant vocation à être livrés en production, j'étais cette fois pleinement intégré aux sprints.

Chapitre 4

Appropriation scientifique : la dette technique et ses enjeux business

4.1 Définition et métaphore

[À rédiger.]

4.2 Typologie de la dette technique

[À rédiger.]

4.3 Enjeux business

Traduire la dette technique en termes d'entreprise revient d'abord à reconnaître une tension difficilement contournable. Du strict point de vue du développeur, le confort absolu dans le traitement de la dette consisterait à suspendre tout développement de nouvelle fonctionnalité *business* pour se consacrer pleinement à l'assainissement du code existant. Cette option est pourtant illusoire : il est impossible d'ignorer la nécessité de continuer à livrer de nouvelles fonctionnalités, ne serait-ce que pour rester compétitif. Le marché du numérique est en effet particulièrement volatil et changeant, et cette contrainte s'impose quelle que soit la taille de l'entreprise. Un gel complet des développements serait, au mieux, financièrement encaissable par les plus grandes structures, au prix d'un impact non négligeable sur leur croissance ; il serait tout simplement rédhibitoire pour celles qui sont incapables de mener de front maintenance du legacy et nouveaux développements.

La dette technique se pose donc rarement comme un problème purement technique : elle relève d'un arbitrage permanent entre le temps consacré à la production de valeur

immédiate — le *time-to-market* des fonctionnalités attendues par le marché — et celui investi dans la soutenabilité du logiciel à long terme : coût de maintenance, risque de régression et fiabilité, cette dernière pesant tout particulièrement pour un opérateur télécom. Toute la difficulté consiste à résorber la dette *sans* renoncer à cette capacité de livraison continue. C'est précisément cet équilibre que les deux entreprises ont eu à construire, de manière empirique, comme le détaille le retour d'expérience qui suit.

4.4 Retour d'expérience : stratégies d'adressage et dimension sociale

Au-delà des leviers strictement techniques, les deux entreprises où j'ai exercé montrent que la manière d'adresser la dette dépend autant de l'état de la base de code que de contraintes économiques — et humaines. Cette section revient sur ce que j'ai pu observer dans chacun de ces contextes.

4.4.1 Une même logique d'adressage incrémental

Face à la tension décrite plus haut, les deux entreprises ont adopté une même réponse : un adressage incrémental de la dette. Plutôt que de geler les développements, il s'agit de partir du code existant et, lorsqu'une fonctionnalité doit être ajoutée à un endroit donné, de profiter de l'intervention pour réécrire le code concerné lorsque cela est possible. Chez Free Mobile, cette logique a résulté d'une évolution de la stratégie, privilégiant cette réécriture « au fil de l'eau » plutôt qu'une reprise de contrôle purement frontale.

4.4.2 Des dettes de nature différente

Si la logique est commune, la dette ne s'exprimait pas de la même façon d'une entreprise à l'autre, ce qui conditionnait la marge de manœuvre disponible.

Chez Free Mobile, la dette tenait surtout à la multiplication de *forks* dotés de spécificités, vraisemblablement créés faute de temps pour produire de nouvelles versions. Le code restait toutefois globalement compréhensible et logique, malgré certains choix architecturaux discutables et une méconnaissance de quelques outils externes — par exemple le recours à des requêtes SQL là où des mécanismes plus adaptés existaient.

Chez Beemo, les contraintes de réécriture étaient bien plus fortes. Le code y est plus cloisonné et, paradoxalement, plus lisible, mais aussi beaucoup plus ancien : écrit à l'origine en C, puis en C++. Pour composer avec ces contraintes, des *patterns* ont été mis en place afin de cloisonner totalement le code *legacy* du code moderne : les nouvelles fonctionnalités sont écrites dans une librairie embarquée puis injectée dans le code historique. Le dévelop-

pement suit alors un cours classique ; lorsqu'une fonctionnalité doit être modernisée en profondeur, elle est réécrite côté moderne, puis « gommée » du code ancien. Cette étape de gommage n'a pas encore été engagée frontalement, afin de préserver une compatibilité permettant le *merge* et le rapatriement de branches *legacy* porteuses de correctifs non encore intégrés.

4.4.3 L'enjeu social du traitement de la dette

L'enjeu qui m'a le plus marqué est toutefois d'ordre social. La phase de reprise de contrôle totale du logiciel s'est révélée très chronophage et, pour être honnête, peu gratifiante pour les développeurs. J'ai ainsi pu observer chez certains collègues une perte de motivation parfaitement compréhensible : si la technique constitue un domaine en soi très intéressant, passer des heures dans du code *legacy* peu agréable à lire, pour n'y greffer que de petits ajouts au sein d'un existant déjà complexe, n'a rien de particulièrement attrayant.

Ce constat n'est pas anecdotique : pour qu'une dette technique soit correctement résorbée, l'enjeu humain — motivation des équipes et reconnaissance d'un travail ingrat — me paraît, au regard de ces deux années d'alternance, tout aussi déterminant que les choix d'outillage ou d'architecture.

4.5 Leviers de réduction

[À rédiger.]

4.6 Grille de lecture des missions

Les missions présentées dans la suite partagent un contexte commun, qui en constitue le fil conducteur : l'absence de tests, qu'ils soient unitaires ou d'intégration. À l'origine, ces logiciels n'en comportaient aucun ; à mon arrivée chez Free Mobile, l'équipe montée par David Desmarest en avait certes introduit quelques-uns, mais la couverture demeurait globalement insuffisante.

Cette dette de test se double d'une connaissance fonctionnelle souvent parcellaire des logiciels appelés à évoluer. La conjonction de ces deux facteurs expose à de forts risques de régression dès qu'une modification est apportée : c'est précisément ce qui justifie la mise en place d'une stratégie de tests et qui rend, plus largement, le développement de nouvelles fonctionnalités difficile. Chacune des tâches qui suivent peut ainsi se lire comme une réponse à une facette de ce constat.

Chapitre 5

Tâche 1 — Outil de duplication de configuration Grafana

5.1 Contexte et dette technique initiale

La supervision de la valorisation s'appuie sur **Grafana**, qui agrège dans des *dashboards* les métriques remontées par le *Valo Engine*. Or cette supervision n'existe pas sur une seule instance : on en retrouve plusieurs — typiquement une par environnement (développement, recette, production) ou par réseau — qui doivent présenter les **mêmes tableaux de bord**. Faire vivre ces instances en parallèle suppose donc de **répliquer** la configuration de l'une vers les autres à chaque évolution.

Cette réplication ne se résume pas à une copie : chaque tableau de bord référence ses **sources de données** (*datasources*) par un **identifiant propre à l'instance**. Le même tableau, copié tel quel d'une instance à l'autre, pointe alors vers des identifiants qui n'y existent pas, et l'affichage se retrouve cassé. La synchronisation manuelle — export, copie, puis correction au cas par cas des identifiants — était donc à la fois **fastidieuse et source d'erreurs** : une dette d'outillage sur la chaîne de supervision elle-même.

5.2 Objectif de la mission

La mission consistait à **reprendre le travail d'un stagiaire**, qui avait amorcé un Proof of Concept (PoC) sur ce sujet, pour en faire un outil exploitable. Le but : **dupliquer l'ensemble des tableaux de bord** d'une instance Grafana vers une autre, puis **réécrire automatiquement** les identifiants de sources de données pour qu'ils correspondent à ceux de l'instance cible. Le cœur du problème est donc un **appariement** : faire correspondre, deux à deux, les sources de données de l'instance d'origine et celles de l'instance de destination,

afin de substituer chaque identifiant par son équivalent.

5.3 Conception et choix techniques

La logique métier est directe : Grafana expose une API HTTP permettant de **lister** les tableaux de bord et les sources de données d'une instance, puis d'y **créer** de nouveaux objets. L'outil interroge l'instance source, construit une **table de correspondance** entre les sources de données des deux instances (appariées par nom), puis parcourt chaque tableau de bord pour y **remplacer** les identifiants avant de le pousser sur l'instance cible.

L'enjeu de conception portait surtout sur l'**interface**. Plutôt qu'une ligne de commande aride, l'outil a été doté d'une **interface en mode texte** (Text User Interface (interface en mode texte) (TUI)) à l'aide du framework Python **Textual**. Ce framework a été une **découverte marquante** : il permet de décrire une interface terminal selon un modèle très proche du développement web — une structure de composants apparentée au *HTML*, mise en forme par des feuilles de style apparentées au *CSS*. On obtient ainsi, dans un simple terminal, une interface interactive (listes, sélection des instances, validation de l'appariement) sans quitter l'environnement de travail des administrateurs.

5.4 Réalisation

Le cœur de l'outil tient dans la réécriture des identifiants : à partir de la table de correspondance construite entre les deux instances, chaque référence à une source de données présente dans un tableau de bord est substituée par son équivalent côté cible.

```
# correspondance : datasource de l'instance source -> datasource de la cible,
# appariées par leur nom (ex. "prometheus-valo" présent sur les deux instances)
correspondance = apparier_par_nom(sources_origine, sources_cible)

def reecrire(dashboard):
    # on substitue chaque identifiant de datasource par son équivalent cible ;
    # un dashboard copié sans cette étape pointerait vers des IDs inexistants
    for panel in dashboard["panels"]:
        ancien = panel["datasource"]["uid"]
        panel["datasource"]["uid"] = correspondance[ancien]
    return dashboard
```

Listing 5.1 – Réécriture des identifiants de sources de données lors de la duplication (extrait illustratif).

5.5 Difficultés rencontrées

S'agissant de ma **première tâche** dans l'équipe, l'enjeu principal n'était pas tant la complexité — le besoin était circonscrit et la mission relativement **rapide** — que la **prise en main** de l'environnement : le mode de travail de l'équipe, le dépôt, et la reprise d'un PoC déjà entamé par un tiers. La principale subtilité technique tenait à la **robustesse de l'appariement** des sources de données, l'appariement par nom supposant une nomenclature cohérente d'une instance à l'autre.

5.6 Résultats et bénéfice business

L'outil a transformé une synchronisation manuelle et faillible en une opération **automatisée et reproductible**, supprimant la classe d'erreurs liée aux identifiants de sources de données mal réécrits. Au-delà du livrable, cette première mission a joué un rôle de **mise en jambe** : prise de contact avec l'équipe et son environnement, et découverte de Textual, un framework réinvesti par la suite. *[À compléter : gain de temps mesuré sur la synchronisation des instances.]*

Chapitre 6

Tâche 2 — Maîtrise du coût de compilation du Valo Engine

6.1 Contexte et dette technique initiale

Le *Valo Engine* est écrit en **C++**, langage dont la compilation est réputée **coûteuse**, en temps de calcul comme en mémoire. Cette lourdeur tient à son modèle historique de compilation, hérité du C : chaque fichier source est compilé en une **unité de traduction** indépendante, qui inclut *textuellement* les en-têtes (*headers*) dont elle dépend. Les **templates** aggravent ce mécanisme : leur définition résidant dans les en-têtes, ils sont **réinstanciés à chaque unité de traduction** qui les inclut. Une même instanciation est ainsi recompilée des dizaines de fois à travers le projet.

Le pic se produit à l'**édition des liens**. L'éditeur de liens doit y rassembler tous les fichiers objets et **dédupliquer** les définitions redondantes d'une même fonction ou instanciation, issues des multiples unités de traduction — une étape dont l'empreinte mémoire explose à mesure que le nombre de symboles croît. En lançant une compilation parallèle exploitant l'ensemble des cœurs d'une machine — de l'ordre de **24** —, la chaîne de compilation (toolchain) **saturait la RAM** précisément à cette phase de liaison, rendant la compilation instable voire impossible à mener à son terme. La dette était ici nichée dans la **chaîne de compilation** elle-même : un atout de productivité (le parallélisme) se retournait en obstacle.

6.2 Objectif de la mission

La mission consistait à étudier la **faisabilité d'un compromis** : exploiter au mieux les cœurs disponibles *sans* provoquer la saturation mémoire qui faisait échouer la compilation.

Autrement dit, **ordonnancer** la compilation pour qu'elle reste **soutenable** en mémoire — en particulier durant l'édition des liens — tout en conservant un parallélisme aussi élevé que le permet le budget RAM de la machine.

6.3 Conception et choix techniques

Le diagnostic a d'abord consisté à **isoler la phase critique**. La compilation des unités de traduction se parallélise sans difficulté majeure ; c'est l'**édition des liens**, beaucoup plus gourmande en mémoire par processus, qui fait diverger la consommation lorsque plusieurs liaisons s'exécutent simultanément. La piste retenue revient donc à **découpler le degré de parallélisme** des deux phases : conserver un parallélisme large pour la compilation, mais **borner** le nombre de tâches d'édition des liens exécutées de front, afin de plafonner le pic mémoire sous la capacité réelle de la machine.

À plus long terme, la **cause racine** — la recompilation répétée des en-têtes et des templates — trouve une réponse dans les **modules** de **C++20**, qui compilent une interface **une seule fois** au lieu de la réanalyser à chaque inclusion. Cette voie n'était toutefois pas exploitable : **le support des modules reste partiel** dans les compilateurs. À titre de repère, sur Ubuntu 24.04 LTS, `g++` 13.3 conserve **C++17** comme standard *par défaut* (`__cplusplus = 201703L`) ; l'adoption de C++20 ou C++23, et de leurs modules, supposerait une migration de la chaîne de compilation (toolchain) qui n'est pas encore une option standard et mûre.

6.4 Réalisation

L'idée se matérialise en **limitant la concurrence de l'étape de liaison** indépendamment de celle de la compilation, de sorte que le pic mémoire cumulé des éditions de liens reste sous le plafond de la machine.

```
# Compilation : on exploite tous les cœurs disponibles (étape peu coû
    teuse en RAM)
NB_COEURS=$(nproc)                          # ex. 24 cœurs

# Édition des liens : étape qui fait exploser la RAM -> on borne sa
    concurrence
# pour que le pic mémoire cumulé reste sous la capacité de la machine
LINK_PARALLELE=4                            # déterminé d'après la RAM par
    link

compiler   --jobs "$NB_COEURS"
lier       --jobs "$LINK_PARALLELE"     # le goulot d'étranglement est
    ici, pas avant
```

Listing 6.1 – Borner le parallélisme de l'édition des liens pour contenir le pic mémoire (extrait illustratif).

6.5 Difficultés rencontrées

La principale difficulté a été de **caractériser le pic mémoire** : la saturation ne survenait qu'à l'édition des liens et seulement sous forte concurrence, ce qui en rendait l'observation peu intuitive — le coût n'est pas réparti uniformément sur la durée de la compilation, mais **concentré** sur sa phase finale. La seconde tenait à la **tension entre vitesse et mémoire** : tout plafonnement de la concurrence de liaison améliore la stabilité au prix d'un allongement du temps de compilation, et le bon réglage dépend étroitement du couple (mémoire disponible, taille des objets à lier).

6.6 Résultats et bénéfice business

En bornant la concurrence de la phase de liaison, la compilation redevient **soutenable** sur les machines de développement : elle peut être menée à son terme sans saturer la RAM, là où une parallélisation naïve la faisait échouer. Au-delà du résultat immédiat, cette mission a été l'occasion de **mettre au jour une dette de fond** — le coût du modèle de compilation C++ préalable aux modules —, dont la résorption complète reste suspendue à la maturation du support de C++20/23 dans les compilateurs. *[À compléter : temps de compilation obtenu et seuil de concurrence retenu.]*

Chapitre 7

Tâche 3 — Framework de test d'intégration et de non-régression

7.1 Contexte et dette technique initiale

La non-régression du *Valo Engine* reposait initialement sur une suite de tests écrits en Lua. Ce langage permettait de scripter des scénarios, mais ne s'avérait pas le mieux adapté à ce que l'on cherchait à faire. Sa nature **impérative** laissait en effet au rédacteur d'un test toute latitude pour y introduire de la logique : un test pouvait dès lors échouer non à cause d'une régression du logiciel testé, mais à cause d'un défaut du test lui-même. La frontière entre code de test et code testé devenait floue, ce qui fragilisait la confiance accordée à ce filet de sécurité. À cela s'ajoutait un couplage de bas niveau : le protocole de communication du moteur — un protocole TCP décrit en C — était embarqué dans le Lua, ce qui rendait la suite délicate à maintenir.

7.2 Objectif de la mission

L'objectif était de remplacer cette suite Lua par un framework dédié, reposant sur des scénarios **déclaratifs**, volontairement plus contraignants. En restreignant fortement la forme des tests — un scénario décrit *quoi* vérifier, jamais *comment* —, on garantit que l'échec d'un test, dès lors que le framework se trouve dans une version stable, provient bien du code du logiciel testé et non de la mécanique de test. Le nouveau framework devait au minimum couvrir le périmètre fonctionnel déjà assuré par le Lua : la simulation des trames d'appel (appels voix, envoi de SMS et de MMS, consommation de data) ainsi que la vérification des données ainsi produites.

7.3 Conception et choix techniques

La première étape a consisté à **porter en Python** le protocole TCP par lequel le framework dialogue avec le *Valo Engine*, jusque-là décrit en C et embarqué dans le Lua. Le module `ctypes` de Python permet de définir des structures dont la disposition binaire est strictement équivalente à celle de leurs homologues C ; on reconstitue ainsi un échange de trames identique à celui du moteur sans recourir au langage natif.

Le framework prend en entrée des scénarios de test rédigés en YAML. Ces scénarios sont analysés par un **parseur SAX**, qui présente deux avantages : il permet de récupérer la position (ligne) de chaque instruction — information transmise au débogueur pour offrir une exécution *pas à pas* — et il interrompt l'analyse dès la première erreur de syntaxe en indiquant sa localisation et sa nature (arguments mal formés ou instruction non supportée).

L'exécution repose sur une **boucle séquentielle** simple : les instructions sont jouées les unes après les autres, sans branchement ni retour arrière. Chaque ligne de texte est transformée en objet *instruction* suivant le patron *Command* : une *factory* instancie la commande correspondante à partir d'un *registry* où sont enregistrées toutes les commandes supportées.

Autour de cette boucle, le framework fournit les services nécessaires pour **moquer** les briques avec lesquelles le logiciel testé — le *Valo Engine* — interagit : un service simulant les équipements qui interrogent le moteur (autorisation d'appel ou de SMS, vérification du crédit selon la localisation) et un service recevant les notifications de seuil émises par le moteur (courriels et messages). Enfin, un ensemble d'instructions de **vérification** contrôle, pour une trame d'appel donnée, la cohérence des données générées en base.

Le mécanisme de vérification reprend le principe de *snapshot* déjà employé côté Lua : les actions jouées ainsi que l'état final de la base de données sont sérialisés, puis comparés à une trace de référence enregistrée lors d'une exécution validée ; toute divergence signale une régression. Pour disposer d'emblée d'une restitution des résultats exploitable — comptes rendus, agrégation des échecs, comparable à celle d'une suite de tests classique — le framework a été bâti comme une **surcouche de Pytest**, la génération et la comparaison des snapshots étant déléguées à la bibliothèque **syrupy**.

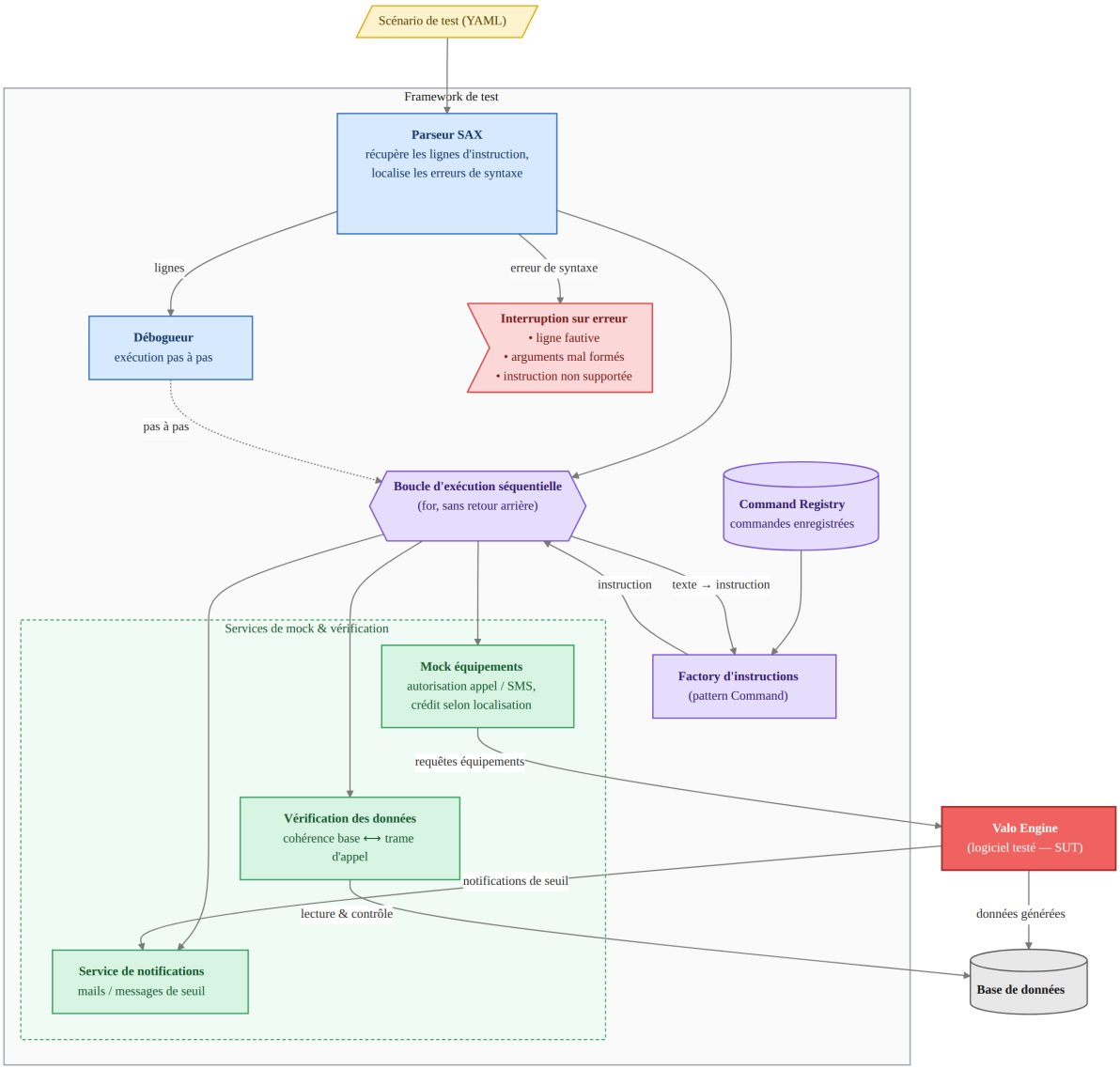


Figure 7.1 – Architecture du framework de test.

7.4 Réalisation

[À rédiger.]

7.5 Difficultés rencontrées

S'agissant de l'une des premières missions d'envergure qui m'étaient confiées, la principale difficulté a tenu au **découpage du travail**. Plutôt que de procéder par versions intermédiaires, livrables et vérifiables au fur et à mesure, je me suis engagé d'emblée dans la conception complète d'une première version couvrant l'ensemble des fonctionnalités du Lua. Ce choix a allongé le délai avant d'obtenir un premier retour concret et m'a privé des points de contrôle qu'un découpage plus fin aurait offerts.

Le second point, apparu avec le recul, concerne le choix de bâtir le framework sur Pytest. Si cet outil fournit gratuitement un socle d'exécution et de reporting éprouvé, il recèle aussi de nombreux mécanismes implicites — sa « *black magic* » — dont le comportement peut se révéler déroutant et compliquer le développement d'une surcouche.

7.6 Résultats et bénéfice business

[À rédiger.]

Chapitre 8

Tâche 4 — Outillage du framework : extension VS Code (LSP et débogueur)

8.1 Contexte et dette technique initiale

Une fois le framework de test livré (tâche 3, chapitre 7), un nouveau frein est apparu : la rédaction des tests elle-même. Les scénarios sont décrits en YAML, support sur lequel l'écriture ne bénéficie d'aucune assistance contextuelle. Là où des langages comme Python, Lua ou C++ disposent d'un serveur de langage (Language Server Protocol (LSP)) offrant autocomplétion et analyse syntaxique au fil de la frappe, le YAML n'est outillé que pour la validation de schémas. Il ne propose pas d'assistance qui tienne compte du contexte — capable, par exemple, de savoir si l'on est en train de saisir une instruction ou l'un de ses arguments.

Or le format de test n'est pas un simple document de configuration : c'est en réalité un langage à part entière, bâti au-dessus du YAML. On y manipule des **instructions** — équivalentes à des fonctions —, des **constantes** ainsi que des **variables** qui constituent les arguments passés à ces instructions. Faute d'outillage tenant compte de cette grammaire, les erreurs de syntaxe n'étaient découvertes qu'à l'exécution et aucun moyen n'existait d'observer pas à pas le déroulé d'un scénario. Cette friction freinait l'adoption du framework par l'équipe : une dette « d'outillage ».

8.2 Objectif de la mission

La mission a consisté à développer une extension *Visual Studio Code*, écrite en **TypeScript**, au service du framework de la tâche 3. Cette extension réunit deux facettes complémentaires :

— un **serveur de langage** (LSP) apportant autocomplétion, diagnostics en temps réel et aide à la saisie. En s'appuyant sur la grammaire du format de test — instructions, constantes et variables —, il fournit une assistance *contextuelle* : il sait distinguer l'endroit où l'on attend une instruction de celui où l'on saisit un argument, et propose les complétions correspondantes ;
— un **débogueur** permettant d'exécuter un test puis, lorsqu'une instruction échoue, d'inspecter l'état du *Valo Engine* au moment précis de l'échec, ou de progresser *pas à pas* pour comprendre le comportement interne du moteur en fonction des trames envoyées par le test.

Une fonctionnalité particulièrement utile en a découlé : la possibilité de lancer le *Valo Engine* en **mode débogage** et de progresser pas à pas *à l'intérieur même de son code C++*, tout en rejouant un scénario d'appel (*call flow*) étape par étape. Le diagnostic ne se limite ainsi plus au test, mais descend jusqu'au code natif du logiciel sous test.

8.3 Conception et choix techniques

Le choix du LSP répond à un principe de **découplage** entre l'éditeur et le langage : la logique d'analyse et de complétion est portée par un serveur indépendant, réutilisable au-delà de VS Code par tout éditeur compatible. Le serveur exploite la grammaire du format de test — celle-là même qu'analyse le parseur du framework (chapitre 7) — pour produire des complétions et des diagnostics cohérents avec ce que le framework acceptera réellement à l'exécution.

Le débogueur, lui, repose sur le **Debug Adapter Protocol** (Debug Adapter Protocol (DAP)), protocole standardisé par VS Code pour piloter une session de débogage au moyen d'échanges *JSON-RPC*. Côté framework, son intégration s'appuie sur un module qui contrôle le flot d'exécution du test — mise en pause, reprise, progression pas à pas — tout en remontant à l'éditeur les données nécessaires au diagnostic.

Les seules données internes au framework de test ne suffisent toutefois pas à diagnostiquer l'ensemble des problèmes : il faut aussi connaître l'état interne du moteur. Un **service** dédié — développé par David — a donc été mis en place pour exposer l'état des variables internes du *Valo Engine*. L'extension dialogue avec ce service via **HTTP** et restitue ces valeurs directement dans l'Integrated Development Environment (IDE), aux côtés de l'état du test.

8.4 Réalisation

Côté LSP, le serveur s'abonne aux requêtes de complétion émises par l'éditeur. À partir de la position du curseur, il détermine le contexte grammatical courant et renvoie la liste

des propositions adaptées — instructions disponibles ou variables déjà déclarées.

```
// Déclenché par l'éditeur à chaque demande de complétion
connection.onCompletion((params) => {
  // On déduit le contexte grammatical à partir de la position du
     curseur
  const contexte = analyserContexte(document, params.position);

  // Selon que l'on attend une instruction ou un argument,
  // on ne propose pas le même jeu de complétions
  return contexte === "instruction"
    ? instructionsSupportees()   // liste tirée du registry du framework
    : variablesDeclarees(document);
});
```

Listing 8.1 – Fourniture d'autocomplétion contextuelle via le LSP (extrait illustratif, TypeScript).

Côté débogueur, l'adaptateur DAP traduit les ordres de l'éditeur (poser un point d'arrêt, exécuter l'instruction suivante) en commandes pilotant le flot d'exécution du test, puis renvoie l'état courant — instruction en cours, état du *Valo Engine* obtenu via le service HTTP.

```
// Requête DAP : l'éditeur transmet les lignes où s'arrêter
protocol.onSetBreakpoints((args) => {
  // Les lignes proviennent du parseur du framework,
  // qui associe chaque instruction à sa position dans le YAML
  moteurDeTest.armerPointsArret(args.lines);

  // On confirme à l'éditeur les points d'arrêt effectivement posés
  return { breakpoints: args.lines.map(verifierLigne) };
});
```

Listing 8.2 – Pas à pas : pose des points d'arrêt côté adaptateur de débogage (extrait illustratif, TypeScript).

8.5 Difficultés rencontrées

La première difficulté a tenu à la **documentation de l'API d'extension de VS Code**. Si des exemples d'extensions existent, l'API elle-même reste inégalement documentée, et aucun standard ne s'impose réellement quant à la manière de structurer le code. Les exemples disponibles sont fréquemment **monolithiques**, ce qui offre peu de modèles d'architecture réutilisables et a imposé un travail d'expérimentation pour aboutir à une organisation maintenable.

La seconde difficulté a porté sur l'**intégration du DAP** au framework de test. Il a fallu adapter le protocole — fondé sur des échanges *JSON-RPC* — à un module capable de

contrôler le flot d'exécution d'un test tout en remontant les données utiles au diagnostic. La restitution de l'état interne du moteur, obtenue par un service HTTP distinct, a par ailleurs supposé de coordonner deux canaux de communication (DAP et HTTP) au sein d'une même session de débogage.

8.6 Résultats et bénéfice business

[À rédiger : temps d'écriture d'un test réduit grâce à l'autocomplétion et aux diagnostics en temps réel, erreurs de syntaxe détectées avant l'exécution, diagnostic accéléré des comportements inattendus — jusque dans le code C++ du moteur — et, in fine, meilleure adoption du framework par l'équipe. À relier au thème de la dette technique, chapitre 4.]

Chapitre 9

Tâche 5 — Portage du framework vers les forks legacy du Valo Engine

9.1 Contexte et dette technique initiale

Le framework de test (tâche 3, chapitre 7) avait été conçu pour le *Valo Engine* du **réseau prépayé** de Free Mobile, un réseau alors nouveau. Une fois ce dernier mis en production et jugé **stable**, la stratégie de gestion de la dette technique a été révisée — une inflexion qui a coïncidé avec le départ de David.

La stratégie initiale faisait du prépayé le socle d'une **base de code commune et moderne**, destinée à être réutilisée par les moteurs de valorisation des différents réseaux : on aurait ainsi migré progressivement chaque réseau vers cette version moderne. Elle a cédé la place à une approche de **quick fix** — que l'on pourrait presque qualifier de *quick converting* —, visant à **stabiliser au maximum** les réseaux existants sans en moderniser le code ; la modernisation n'interviendrait, le cas échéant, qu'une fois cette stabilisation acquise.

Or ces réseaux reposent sur autant de fork *legacy* du moteur, chacun doté de ses spécificités métier. Plusieurs inconnues pesaient sur les exécutables réellement en production : certains binaires n'avaient pas été mis à jour depuis longtemps, et les sources se répartissaient entre le dépôt GitLab et diverses machines. Toucher à ce code sans filet était d'autant plus risqué que le *Valo Engine* calcule le **coût de chaque appel** : une régression ou une panne sur ce maillon peut engendrer des pertes **de l'ordre du million d'euros en une semaine**. La dette tenait donc à l'absence de tout moyen de garantir qu'un correctif n'introduise pas, par inadvertance, un **changement de logique métier** sur l'un de ces forks.

9.2 Objectif de la mission

La mission a consisté à **porter le framework de test** vers l'ensemble des forks legacy du *Valo Engine*, afin de doter chaque réseau d'un filet de non-régression *avant* d'y appliquer le moindre correctif. Le périmètre couvrait six variantes : la France **B2C** et **B2B**, les **Caraïbes**, l'Italie **B2C** et **B2B**, ainsi que le **proxy** italien.

L'objectif premier n'était pas de couvrir fonctionnellement chaque réseau dans les moindres détails, mais de disposer d'un **contrôle sur le code métier** de chaque fork : pouvoir y appliquer des correctifs avec l'assurance que la logique de valorisation demeurait inchangée.

9.3 Conception : factorisation et paramétrage

La principale difficulté de conception tenait à l'hétérogénéité des réseaux, qui se répartissent en deux modèles de facturation. Les réseaux en online charging fonctionnent comme le prépayé : les trames d'appel, de SMS et de data sont échangées en temps réel via un protocole TCP interne. Les réseaux en offline charging, eux, ne dialoguent pas en direct avec le moteur : celui-ci **consomme des CDR**, c'est-à-dire des fichiers décrivant a posteriori la consommation. Un premier chantier a donc consisté à doter le framework de la capacité à **produire et lire des CDR** pour simuler les équipements en *offline charging*, là où il ne savait jusque-là qu'émettre des trames TCP.

Pour absorber cette diversité, David avait amorcé une refonte du framework articulée autour de deux notions complémentaires :

- un **core** — le socle commun à toutes les versions supportées. Il porte la mécanique invariante : la boucle d'exécution et le *parsing* des instructions du langage de test ;
- des **specs** — une par fork — qui spécifient *quelles* instructions sont disponibles et *comment* elles s'exécutent, en fournissant leur **implémentation runtime** propre au réseau visé.

Cette séparation *core* / *spec* permet de mutualiser tout ce qui ne dépend pas du réseau et de n'isoler, fork par fork, que les différences de comportement métier — préparant du même coup la modernisation ultérieure.

9.4 Réalisation

Le portage s'est déroulé en **binôme**. Ayant déposé ma démission, je m'inscrivais dans une démarche de **transmission de connaissances** avec Jérémy, recruté en renfort de l'équipe. Travailler à deux lui a permis de s'approprier un framework devenu conséquent — de l'ordre de 30 000 lignes, près de 40 000 en comptant le code généré pour la communication TCP — tout en menant le portage à bien.

Concrètement, chaque fork a été décrit par sa *spec*, qui enregistre auprès du *core* l'implémentation des instructions propres au réseau ; le *core* reste, lui, identique d'un fork à l'autre.

```
# Le core fournit la boucle d'exécution et le parsing, communs à tous les forks.
# Chaque spec n'apporte que ce qui diffère : l'implémentation runtime des
# instructions et le mode de communication avec l'engine (TCP ou CDR).
class SpecItalieB2C(Spec):
    charging = OfflineCharging()       # consommation de CDR (fichiers)

    @instruction("appel_voix")
    def appel_voix(self, ctx, args):
        cdr = self.charging.encoder_cdr(args)    # encodage au format du fork
        ctx.engine.consommer(cdr)                # alimente le Valo Engine legacy
```

Listing 9.1 – Paramétrage d'un fork : une *spec* enregistre ses instructions auprès du *core* (extrait illustratif).

9.5 Difficultés rencontrées

Deux obstacles ont dominé la réalisation. Le premier a été le **décodage du format binaire des CDR**. Ces enregistrements sont certes normalisés en amont par une brique dédiée, le *CDR hub* ; il restait néanmoins à **décoder leur contenu** : la disposition exacte des données, la valeur de chaque champ et, plus encore, ce que ces champs représentent d'un point de vue fonctionnel.

Le second a tenu au **pilotage des engines legacy**. Contrairement au moteur du prépayé, beaucoup plus moderne et doté d'un service de testing exposant une **API HTTP**, les forks legacy n'avaient **jamais été pensés pour accueillir des tests**. Il a donc fallu trouver le moyen de les **démarrer**, de les **arrêter** et de **dialoguer** avec eux, sans le point d'entrée dédié dont bénéficiait le prépayé.

9.6 Résultats et bénéfice business

Le portage a étendu le filet de non-régression aux forks legacy du *Valo Engine* : les correctifs de stabilisation pouvaient désormais être appliqués avec l'assurance qu'aucun **changement de logique métier** ne s'y glissait à l'insu de l'équipe. Sur un composant dont une défaillance se chiffre en millions d'euros par semaine, ce contrôle constitue avant tout une **maîtrise du risque financier**.

Au-delà du gain immédiat, l'architecture *core* / *spec* factorise la mécanique commune et n'isole que les spécificités de chaque réseau : elle réduit la duplication entre forks et **prépare la modernisation** envisagée à plus long terme (chapitre 4).

Chapitre 10

Tâche 6 — Rétro-ingénierie pour la compilation avec un SDK

10.1 Contexte et dette technique initiale

Cette tâche est sensiblement plus **courte** que celles menées chez Free Mobile, mais elle n'en est pas moins **déterminante** dans la démarche de réduction de la dette technique. Elle porte sur l'un des logiciels de Beemo, **l'agent VSX**, et vise à reprendre le contrôle sur une partie de sa **chaîne de production**. Sa portée dépasse le simple correctif : il s'agit d'**automatiser** et d'**harmoniser** la méthode de fabrication de cet agent — ce qui la rend, à mes yeux, aussi structurante que la réalisation du framework de test étalée sur près d'un an et demi.

Avant cette mission, mettre à jour l'agent VSX supposait une manipulation manuelle et fragile : on **poussait la base de code vers une machine distante**, sur laquelle elle était compilée avec le **DSDK** — le Software Development Kit (SDK) fourni par Broadcom pour la réalisation de *daemons* (processus d'arrière-plan) —, avant de pouvoir être testée.

La dette technique est apparue avec le **passage à la version 8 du DSDK**, nécessaire pour supporter la version 8 de l'agent. Dans sa configuration de base, le DSDK impose le flag de compilation C++ `-Wall -Werror`, qui **convertit l'ensemble des avertissements en erreurs** bloquantes. Or l'agent est écrit en C++11, avec une **absence quasi totale de bonnes pratiques** de génie logiciel — certains fichiers dépassent les **50 000 lignes**. Le volume d'avertissements produit est dès lors **colossal** et intraitable en l'état : il met un frein considérable à la capacité de l'équipe à **livrer des mises à jour en continu**.

Une rustine existait bien — des modifications appliquées directement sur la machine de compilation pour neutraliser ce comportement — mais elle n'était **ni reproductible ni**

documentée. Ce savoir reposait sur le développeur **que je remplace** : son départ menaçait de l'emporter avec lui.

10.2 Objectif de la mission

La mission a poursuivi deux objectifs complémentaires :

- **s'affranchir de la machine de compilation distante** en rendant le *build* reproductible localement, à l'aide d'un conteneur **Docker** embarquant la chaîne de compilation (toolchain) du DSDK ;
- **rendre les correctifs du SDK reproductibles et documentés**, en substituant aux modifications manuelles des **fichiers de patch** versionnés, appliqués automatiquement lors de la construction de l'image.

Atteindre ces deux objectifs supposait au préalable de **comprendre le fonctionnement interne du DSDK** — une démarche de rétro-ingénierie, le SDK n'exposant aucun moyen standard d'agir sur ses flags de compilation.

10.3 Démarche de rétro-ingénierie

Le cœur de la tâche a consisté à **retrouver où et comment le DSDK injecte ses flags de compilation**. Cette mécanique n'est **pas exposée** par le SDK : il n'existe aucune option officielle pour assouplir le couple `-Wall -Werror`. Il a donc fallu **analyser le système de build du DSDK lui-même** pour localiser le point précis où ces flags sont posés, puis déterminer comment les neutraliser sans dénaturer le reste de la chaîne de compilation (toolchain).

La difficulté principale n'a pas été d'identifier la modification à apporter, mais de **trouver le bon point d'injection** : le DSDK **n'est pas conçu pour être modifié**, et un patch mal placé compromet l'intégrité de la chaîne. Une fois ce point localisé, la suite s'est révélée nettement plus simple : il a suffi de cibler les fichiers concernés et d'en dériver des correctifs reproductibles.

10.4 Réalisation

La solution repose sur un **Dockerfile** qui reconstitue l'environnement de compilation auparavant confiné à la machine distante, puis applique les correctifs du SDK de façon déterministe. Chaque modification est isolée dans un fichier `.patch` versionné et appliquée via la commande `patch` lors de la construction de l'image — là où elle était jusqu'alors saisie à la main, sans trace.

```
1 # Image de compilation embarquant la toolchain Broadcom (DSDK v8),
```

```
# auparavant disponible uniquement sur une machine distante.
FROM broadcom/dsdk:8

# Application reproductible des correctifs du SDK : chaque .patch est
# versionne et documente, en lieu et place des modifications manuelles
# faites jusqu'ici directement sur la machine de compilation.
COPY patches/ /tmp/patches/
RUN patch -p1 -d "${DSDK_ROOT}" < /tmp/patches/0001-assouplir-werror.
    patch

# Le build de l'agent VSX devient ainsi local et rejouable a l'identique
    .
WORKDIR /src
RUN make agent-vsx
```

Listing 10.1 – Reconstitution de la toolchain et application reproductible des correctifs du DSDK dans le Dockerfile (extrait illustratif).

10.5 Difficultés rencontrées

La principale difficulté a tenu à l'**opacité du DSDK** et à l'identification du point d'injection des patchs, évoquée plus haut. Une seconde est apparue en aval : l'un des correctifs appliqués au SDK a eu un **effet de bord sur la certification** de l'un des *packages* générés, qui n'est plus accepté en l'état. C'est la raison pour laquelle la tâche demeure **encore en cours** : le *build* local et reproductible est acquis, mais la **validation du package** reste à fiabiliser.

10.6 Résultats et bénéfice business

La mission a transformé une **procédure manuelle, distante et non documentée** en un *build* **local, reproductible et versionné**. Les correctifs du SDK, jadis tributaires d'un seul développeur, sont désormais **capitalisés sous forme de patchs** : leur logique survit à tout départ, ce qui répond directement au risque de **perte de connaissance** à l'origine de la dette. En supprimant le frein imposé par `-Wall -Werror`, l'équipe retrouve sa capacité à **livrer des mises à jour de l'agent VSX en continu**.

Le chantier restant — résoudre l'impact sur la certification — s'inscrit dans une perspective plus large : **automatiser le processus de validation d'un package par Broadcom**, aujourd'hui **très chronophage**. À terme, c'est l'ensemble de la chaîne de production de l'agent qui gagnerait en **harmonisation** et en **autonomie**, prolongeant la résorption de la dette bien au-delà du seul déblocage de la compilation.

Chapitre 11

Tâche 7 — Développement d'une fonctionnalité en code de production

11.1 Contexte et besoin

Le métier de Beemo n'est pas de proposer un **service de cloud** à la manière des grands fournisseurs — Amazon, Google ou Microsoft. Il s'agit de fournir aux **professionnels** — hébergeurs et entreprises pour qui la donnée constitue un actif critique — un moyen de **sécuriser leurs données** et de garantir leur **restauration** en cas d'incident, qu'il soit d'origine **physique** ou **malveillante**. Le produit central, la **BBox**, est une appliance de **sauvegarde** qui peut être **externalisée** hors site afin de survivre à la destruction du système d'origine.

L'un des scénarios de perte de données les plus redoutés est le **ransomware**, qui chiffre les données d'un système pour en exiger une rançon. La BBox répond directement à cette menace en conservant des **jeux de sauvegarde** hors d'atteinte du système compromis. Mais une question demeure : **que se passe-t-il si la BBox elle-même est compromise** ? Un attaquant parvenu jusqu'à l'appliance pourrait alors tenter d'**altérer les jeux de sauvegarde** ou de **saboter leur restauration** — anéantissant la dernière ligne de défense du client.

C'est pour répondre à ce scénario que Beemo développe une nouvelle fonctionnalité, l'**immuabilité**. Son principe : rendre un jeu de sauvegarde **insupprimable** et **inaltérable** tant qu'un **délai de rétention** n'est pas écoulé, y compris pour un utilisateur disposant de droits sur la box — et donc pour un attaquant qui en aurait pris le contrôle. J'ai participé à son développement aux côtés de **deux autres développeurs**, en me concentrant sur la partie **embarquée sur la box**, écrite en **C++**.

11.2 Objectif de la mission

L'objectif n'était pas seulement de **livrer la fonctionnalité**, mais de le faire dans un contexte particulièrement défavorable : celui d'une base de code embarquée portant une **dette technique considérable**. L'intérêt de cette tâche, au regard du fil rouge de ce mémoire, tient précisément là — **comment injecter une fonctionnalité moderne dans du code dégradé sans en briser l'existant** ?

Le firmware de la box cumule en effet la plupart des symptômes de la dette :

- un **abus de directives de préprocesseur** (`#ifdef`, `#ifndef`) pour gérer les multiples cas de **compilation** de la box, rendant le code difficile à lire et à raisonner ;
- des **fonctions extrêmement longues** et une **maîtrise approximative du C++**, donnant lieu à des constructions trompeuses — par exemple des contrôles du type `this == nullptr`, un cas qui **ne peut jamais se produire** et trahit une incompréhension du langage ;
- une **absence totale de tests** — ni test de non-régression, ni tests unitaires — qui prive de tout filet de sécurité lors d'une modification ;
- un fichier `main.h` qui **inclut l'ensemble des fichiers** du projet, créant un **couplage massif** : réutiliser une fonctionnalité existante depuis du code neuf devient une opération complexe et risquée.

La contrainte centrale était donc de **ne pas aggraver cette dette** : il fallait écrire du code **moderne, testable et isolé**, tout en **dialoguant** avec un existant qui ne s'y prête pas.

11.3 Conception et intégration

La réponse à cette tension repose sur une **couche intermédiaire** d'**adaptation**, conçue par David, l'un des développeurs de l'équipe. L'idée est d'**encapsuler** une fonctionnalité — qu'elle soit **legacy** ou **nouvelle** — derrière une **classe** pouvant être appelée **indifféremment depuis l'ancien code ou depuis le code neuf**. Les appels vers l'existant sont **masqués** par des **pointeurs** transmis à cette classe, ce qui **découple** le nouveau code du couplage imposé par `main.h`.

Ce mécanisme d'*enrobage* (*wrapping*) joue le rôle de **frontière** : d'un côté, le firmware historique et ses contraintes ; de l'autre, un espace où l'on peut écrire du **C++ réellement moderne** pour réaliser la fonctionnalité. La fonctionnalité d'immuabilité a ainsi pu être bâtie autour de deux briques propres :

- un **automate à états** (*state machine*) qui pilote le cycle de vie d'un jeu de sauvegarde immuable — création, période de rétention, mise en pause du **planificateur de sauvegarde**, expiration ;
- un **service de chiffrement** qui garantit l'**intégrité** des données. La base est **lue et**

déchiffrée à l'aide d'une clé : si elle a été **altérée**, le déchiffrement échoue, **le code le détecte** et la box se **bloque** — empêchant toute manipulation des jeux de données réels par un attaquant parvenu jusqu'à l'appliance.

L'immuabilité vient ainsi s'ajouter aux mécanismes de sécurisation existants : elle **empêche la suppression** d'un jeu de sauvegarde considéré comme immuable tant que le **délai de rétention** n'a pas été respecté.

11.4 Réalisation

La couche d'adaptation est le point d'entrée de toute la fonctionnalité. L'extrait ci-dessous en illustre le principe : une classe moderne expose le service d'immuabilité au reste du système, tout en dissimulant les appels au firmware historique derrière un pointeur opaque transmis à la construction.

```
// La fonctionnalite d'immuabilite est ecrite en C++ moderne, isolee du
// firmware historique. Le couplage impose par main.h est neutralise :
// le code neuf ne connait l'existant qu'a travers un pointeur opaque.
class ImmutabilityService {
public:
    // Le legacy est injecte par pointeur a la construction : le code neuf
    // n'inclut jamais main.h et reste testable de maniere isolee.
    explicit ImmutabilityService(LegacyBackupContext* ctx) noexcept
        : m_ctx(ctx) {}

    // Verrouille un jeu de sauvegarde : il devient insupprimable et
    // inalterable tant que le delai de retention n'est pas ecoule.
    Result lock(const BackupSetId& id, RetentionDelay delay) {
        // L'automate a etats pilote le cycle de vie de l'immuabilite.
        if (!m_state.canTransitionTo(State::Locked))
            return Result::Rejected;

        // L'integrite est garantie par chiffrement : a la relecture, un
        // echec de dechiffrement revele toute alteration de la base.
        auto sealed = m_cipher.seal(id, delay);
        m_state.transition(State::Locked);
        return sealed ? Result::Ok : Result::IntegrityError;
    }

private:
    LegacyBackupContext* m_ctx;   // pointeur opaque vers le firmware legacy
    ImmutabilityStateMachine m_state;
    CipherService m_cipher;
};
```

Listing 11.1 – Couche d'adaptation isolant le code neuf du firmware legacy : les appels à l'existant sont masques derriere un pointeur (extrait illustratif).

11.5 Difficultés rencontrées

La principale difficulté n'a pas été la **conception de la fonctionnalité** en elle-même, mais sa **cohabitation avec l'existant**. Comprendre le firmware historique — afin de **réutiliser ses fonctionnalités** depuis le code neuf — s'est avéré coûteux, en raison du **couplage par** `main.h`, des **fonctions interminables** et des **cas de compilation** multipliés par les directives de préprocesseur.

L'**absence de tests** a constitué un risque permanent : sans test de non-régression, la moindre modification dans une zone partagée pouvait **casser silencieusement** une fonctionnalité existante. La couche d'adaptation a ici joué un rôle de **garde-fou**, en **circonscrivant** les points de contact avec le legacy et en rendant le code neuf **testable de manière isolée** — là où le reste du firmware ne l'est pas.

11.6 Résultats et bénéfice business

Cette tâche illustre une facette essentielle de la lutte contre la dette technique : il ne s'agit pas toujours de **résorber** l'existant, mais parfois de savoir **construire proprement à côté** sans l'aggraver. Grâce à la couche d'adaptation, la fonctionnalité d'immuabilité a pu être développée en **C++ moderne**, **isolée** et **testable**, alors même qu'elle s'insère dans un firmware fortement dégradé et dépourvu de filet de sécurité.

Sur le plan **business**, l'immuabilité renforce directement la **proposition de valeur** de Beemo : elle protège les jeux de sauvegarde non seulement contre la compromission du système client, mais aussi contre la **compromission de la BBox elle-même**. En garantissant qu'un jeu de sauvegarde immuable ne peut être ni supprimé ni altéré avant l'expiration de son **délai de rétention**, et en **bloquant** la box au moindre signe d'altération détecté par le chiffrement, la fonctionnalité ferme l'un des derniers angles d'attaque possibles — consolidant la promesse fondamentale du produit : **la donnée sauvegardée n'est jamais perdue**.

Chapitre 12

Conclusion

12.1 Bilan fonctionnel

[À rédiger.]

12.2 Retour sur le fil rouge : la dette technique

[À rédiger.]

12.3 Apports de l'alternance

[À rédiger.]

12.4 Perspectives

[À rédiger.]

Glossaire des acronymes

API Application Programming Interface.

CD Continuous Delivery / Deployment.

CDR Call Detail Record (enregistrement détaillé d'appel).

CI Continuous Integration (intégration continue).

DAP Debug Adapter Protocol.

IDE Integrated Development Environment.

LSP Language Server Protocol.

PoC Proof of Concept.

SDK Software Development Kit.

TUI Text User Interface (interface en mode texte).

Glossaire

chaîne de compilation (toolchain) Ensemble des outils (compilateur, éditeur de liens, bibliothèques) permettant de transformer le code source en binaire exécutable pour une cible donnée.

dette technique Coût implicite des retouches futures induit par le choix d'une solution rapide plutôt que d'une approche meilleure mais plus longue à mettre en œuvre. Se rembourse par le refactoring ; génère des « intérêts » sous forme de ralentissement du développement.

fork Divergence d'une base de code en plusieurs variantes maintenues en parallèle, partageant une origine commune.

offline charging Modèle de facturation a posteriori : la consommation est décrite dans des fichiers (CDR) que le moteur de valorisation consomme après coup, sans dialogue en temps réel avec les équipements.

online charging Modèle de facturation en temps réel : les équipements interrogent le moteur de valorisation pendant l'appel (ou l'envoi de SMS/data) via un protocole d'échange direct, ici un protocole TCP interne.

rétro-ingénierie Analyse d'un système ou binaire existant pour en comprendre la structure et le comportement, en l'absence de documentation ou de sources complètes.

test de non-régression Test vérifiant qu'une modification du code n'a pas réintroduit de défaut ni cassé une fonctionnalité existante.

Bibliographie

Annexe A

Annexes

A.1 Diagramme de Gantt

[À insérer.]

A.2 Extrait de code choisi

[Facultatif.]